

自竞流：面向开放交通交互的自博弈强化学习自动驾驶框架

万宏飞^a

^a 清华大学工程物理系, , 北京, 中国

Abstract

本文提出 自竞流 (Selfrace) 框架, 面向开放交通交互场景中的自动驾驶决策学习问题, 构建一个以自博弈强化学习为核心的多智能体仿真与训练系统。与依赖固定人类轨迹数据或手工规则交通流的方法不同, 本工作让同一策略网络同时控制大量交通参与者, 在隐藏目标、有限观测和随机化驾驶偏好的条件下进行闭环交互, 从而学习防御性、可泛化的驾驶行为。系统采用 GPU 批量化仿真, 基于 CARLA 地图预处理构建道路拓扑和局部几何观测, 支持最多 150 个智能体的同场交互; 车辆动作采用 12 个离散 jerk 控制, 动力学由运动学自行车模型推进; 奖励函数显式引入 10 维可随机采样的条件参数, 使同一策略能够表达不同风险偏好、舒适性偏好和车道保持偏好。策略网络使用排列不变编码器处理邻居车辆、车道点和道路边界等集合输入, 并结合 PPO、GAE、DDP 和优势过滤进行训练。本文进一步提出后续封闭场地低速实车验证计划, 用于评估仿真策略在轨迹跟踪、避障、停车、简单交互和安全降级方面的可迁移性。

Keywords: 自博弈强化学习, 自动驾驶, 多智能体仿真, 条件化策略, PPO, 实车验证

1. 引言

自动驾驶系统的难点并不只在于单车沿规划路线行驶, 更在于开放交通环境中的交互决策。真实道路上, 其他车辆、行人和骑行者的目标、偏好、注意力状态和未来动作通常不可见; 一个看似简单的变道、让行或无保护转弯, 都会受到周围参与者行为的强烈影响。传统方法常通过规则车流、日志重放或监督学习数据集构建训练环境, 但这些方式要么行为模式有限, 要么难以覆盖长尾交互场景。

自竞流 的出发点是: 让交通参与者在仿真中互为环境, 通过自博弈产生复杂交互数据。所有智能体共享同一个条件化策略网络, 但每个智能体拥

有独立目标、局部观测和随机化奖励偏好。由于当前智能体无法观察其他参与者的目标和偏好，它必须在交互中学习保守、鲁棒且可恢复的驾驶策略。

大规模自博弈自动驾驶已有工作表明，在无需人类驾驶轨迹训练的情况下，批量化仿真和共享策略训练可以形成自然且鲁棒的驾驶行为 [1]。本文不将该工作作为复述对象，而是以 Selfrace 项目为主体，给出自竞流的系统设计、代码实现细节和后续实车验证路线。本文贡献概括如下：

- 构建面向多智能体自博弈的自动驾驶仿真框架，将地图、空间哈希、动力学、碰撞检测、观测生成、奖励计算和路径规划统一在批量张量流程中；
- 设计结构化观测和条件化奖励输入，使策略能够同时处理道路几何、邻居车辆、目标路径和驾驶风格；
- 实现离散 jerk 动作空间与运动学自行车模型，兼顾控制平滑性、物理约束和强化学习离散动作训练效率；
- 给出从仿真到低速封闭场地实车测试的验证计划，为后续 sim-to-real 迁移和安全评估奠定基础。

2. 自竞流 方法框架

2.1. 多智能体自博弈建模

自竞流 将交通系统建模为部分可观测多智能体随机博弈。每个环境中包含多个智能体，每个智能体独立接收局部观测、选择动作并获得奖励。策略可表示为

$$\pi(a \mid S, W, A, G, C), \quad (1)$$

其中 S 是自车状态， W 是局部道路和交通控制信息， A 是附近动态参与者， G 是路径目标， C 是条件化参数。与完全信息博弈不同，智能体只能观察自己的条件参数，不能直接获知其他车辆的目标、奖励权重或未来路线。

这种设定使训练环境天然包含不确定性：同一场景下，有的车辆可能更重视安全距离，有的车辆可能更激进地追求速度，有的车辆可能对车道中心或舒适性约束不敏感。共享策略在这些差异化智能体之间反复交互，学习到的不再是单一规则，而是一族由条件参数调制的驾驶行为。

2.2. 共享策略与条件化驾驶风格

所有可学习交通参与者共用同一个神经网络。共享策略带来两个好处：第一，系统只需要一次大批量前向推理即可为所有智能体生成动作，利于 GPU 吞吐；第二，来自所有车辆的经验都反向更新同一策略，使训练数据利用率更高。

条件化参数 C 是自竞流的关键设计。当前实现中，奖励条件 C_{reward} 为 10 维，包括目标到达半径、碰撞惩罚、离路惩罚、舒适性惩罚、车道朝向、速度朝向、车道中心、中心偏置、倒车惩罚和停止线惩罚。训练时随机采样这些参数，相当于在同一策略内生成不同驾驶风格；测试或部署时则可以固定一组偏安全、偏舒适或偏效率的参数。

3. 仿真系统设计

3.1. 总体结构

Selfrace 的核心仿真器为 `TeraflowSimulator`。该模块将道路网络、空间哈希、运动学模型、离路检测、碰撞检测、世界初始化、观测生成、奖励计算和路径规划整合为统一的 step 接口。一次仿真步接收形状为 (B, M) 的动作张量，其中 B 为并行环境数， M 为每个环境的最大智能体槽位数；随后系统批量更新所有车辆状态，计算碰撞、离路、Frenet 坐标、局部路径、观测、奖励和 done 标记。

当前默认配置使用 100 个并行环境、每个环境最多 150 个智能体、仿真步长 0.3s。这一规模小于大规模工业训练设置，但更适合本地开发、调试和课程化扩展。后续可以在相同张量接口下提高环境数和 GPU 数量。

3.2. CARLA 地图预处理与道路表示

项目使用 CARLA Town01、Town02、Town03 和 Town10HD 等地图数据，并将其预处理为 JSON 形式的道路结构。道路表面被组织为可查询的四边形、中心线和相关控制信息。地图目录中保留了 stitched map、cross data、traffic controls 以及多种可视化脚本，支持从 CARLA/OpenDRIVE 风格地图到轻量道路图结构的转换。

这种表示服务于三类运行时任务：第一，判断车辆是否在可行驶区域；第二，为每个智能体查询附近车道点和边界点；第三，支持从起点 quad 到目标 quad 的路径规划。通过将道路几何预处理为可张量化查询的数据结构，仿真器避免在训练循环中反复解析复杂地图文件。

3.3. 空间哈希、离路与碰撞检测

SpatialHash 是自竞流的关键加速结构。系统将地图平面划分为固定尺寸网格，默认网格单元为 5 米。静态道路元素和动态车辆包围盒都可以映射到网格单元，从而将全局几何查询转化为局部候选查询。

碰撞检测采用宽阶段加窄阶段。宽阶段通过空间哈希筛选共享网格的候选车辆对，避免 $O(M^2)$ 的全量两两检测。窄阶段把车辆矩形顶点变换到对方局部坐标系中，检查运动线段是否与对方轴对齐包围盒相交。该过程只进行碰撞检测，不模拟碰撞后的物理反弹，因此更适合作为强化学习中的终止和惩罚信号。

离路检测由 **OffroadChecker** 与道路网络配合完成。仿真器在每一步提取车辆位置、航向、长度和宽度，判断车辆是否仍位于道路区域。碰撞和离路共同构成 episode 终止条件的一部分。

3.4. 世界初始化与路径规划

WorldInitializer 负责在道路四边形中心线上批量采样车辆状态。每辆车包含位置、航向、速度、车长、车宽和 active 标记。初始化后，系统通过离路检测和碰撞检测剔除无效车辆，并记录每个智能体的起始 quad id。

路径规划由 **PathPlanner** 完成。仿真器为每个激活智能体随机采样目标 quad id，并生成从起点到目标的路径点序列。当前实现还包含路径观测课程化机制：**path_observation_length** 初始较短，默认从 2 个路径点开始，可逐步放长。这样可以使策略先学习局部跟车、避碰和车道保持，再逐渐学习更长程的路线决策。

4. 观测、动作与动力学建模

4.1. 局部观测

当前自竞流的观测张量由四部分组成：

- 自车状态：7 维，包括位置、航向、速度、车长、车宽和 active 标记；
- 邻居车辆：最多 20 个邻居，每个 7 维，包括相对位置、相对速度、尺寸和 active 标记；
- 车道点：25 个局部车道点，每个 2 维；
- 道路边界点：26 个局部边界点，每个 2 维。

ObservationGenerator 会把邻居车辆、车道点和边界点转换到 ego 坐标系。邻居速度以相对速度形式输入，地图点以自车为中心旋转平移。有限邻居数量和局部视野天然模拟了部分可观测性，迫使策略在信息不完整时保持防御性。

4.2. 离散 *jerk* 动作空间

系统使用 12 个离散动作控制纵向和横向 *jerk*:

$$\dot{a}_{\text{long}} \in \{-15, -4, 0, 4\}, \quad \dot{a}_{\text{lat}} \in \{-4, 0, 4\}. \quad (2)$$

动作索引首先映射为 $(\dot{a}_{\text{long}}, \dot{a}_{\text{lat}})$, 再积分为纵向加速度和横向加速度。离散动作便于使用分类策略和 PPO 训练, 同时 *jerk* 控制比直接控制速度或转角更容易生成平滑轨迹。

4.3. 运动学自行车模型

KinematicBicycleModel 使用批量化运动学自行车模型推进车辆状态。默认物理约束包括: 最大纵向加速度 2.5 m/s^2 、最小纵向加速度 -5.0 m/s^2 、横向加速度范围 $\pm 4.0 \text{ m/s}^2$ 、速度范围 $[-2, 20] \text{ m/s}$ 、最大转向角 35° 、最大转向角变化率 0.6 rad/s 、轴距 2.9 m 。

模型先由 *jerk* 更新加速度, 再用梯形积分更新速度。若加速度或速度跨过零点, 系统将对对应量置零, 以减少符号翻转导致的抖动。横向加速度进一步转换为目标转向角, 并受到转向角变化率限制。最后, 车辆沿有效曲率对应的圆弧更新位置和航向。

项目中还实现了 **DrivingStyleSampler**, 用于采样 C_{throttle} 、 C_{steer} 、 C_{acc} 和 C_{vel} 。当前网络中 4 维车辆风格参数仍预留为零, 后续可将真实动力学随机化参数接入策略输入, 使同一网络适配不同车辆响应。

5. 奖励函数与策略网络

5.1. 奖励函数

自竞流的奖励函数由多个项相加构成:

$$R = R_{\text{goal}} + R_{\text{collision}} + R_{\text{off-road}} + R_{\text{comfort}} + R_{\text{lane}} + R_{\text{velocity}} + R_{\text{reverse}} + R_{\text{stop-line}} + R_{\text{timestep}}. \quad (3)$$

目标奖励鼓励车辆到达路径点或终点; 碰撞、离路、倒车和停止线违规产生惩罚; 舒适性项惩罚过大的加速度和 *jerk*; 车道对齐与车道中心项鼓励车辆沿合理方向、靠近车道中心行驶; 速度项鼓励有效前进; 时间步惩罚抑制无意义动作。

每个 episode 中, 系统为每个智能体采样一组奖励系数。表 1 给出当前实现中的 10 维条件输入。

Table 1: 当前实现中的奖励条件参数。

参数	采样范围	含义
δ_{goal}	$U(2, 12)$	目标到达半径
$\alpha_{\text{collision}}$	$U(0, 3)$	碰撞惩罚强度
α_{boundary}	$U(0, 3)$	离路惩罚强度
α_{comfort}	$U(0, 0.1)$	舒适性惩罚强度
$\alpha_{\text{l-align}}$	$U(2.5 \times 10^{-4}, 2.5 \times 10^{-2})$	车道朝向
$\alpha_{\text{vel-align}}$	$U(0, 1)$	速度方向耦合
$\alpha_{\text{l-center}}$	$U(2.5 \times 10^{-4}, 7.5 \times 10^{-3})$	车道中心
$\alpha_{\text{center-bias}}$	$U(-0.5, 0.5)$	中心偏置
α_{reverse}	$U(2.5 \times 10^{-4}, 7.5 \times 10^{-3})$	倒车惩罚
$\alpha_{\text{stop-line}}$	$U(0, 1)$	停止线违规

5.2. 特征编码与策略网络

网络输入由简单特征和集合特征拼接而成：

$$[S(t), G(t), C_{\text{reward}}, C_{\text{style}}, W_{\text{boundary}}, W_{\text{lane}}, W_{\text{stop}}, A(t)]. \quad (4)$$

当前配置中，简单特征维度为 $[7, 256, 10, 4]$ ，集合特征维度为 $[52, 50, 20, 140]$ ，总输入维度为 539。

普通向量由 `SimpleFeatureEncoder` 编码。边界点、车道点、停止线和邻居车辆由 `PermutationInvariantEncoder` 编码：先对集合中每个元素使用共享 MLP，再沿元素维度做 max pooling。这一设计与 Deep Sets 思想一致 [2]，保证集合输入顺序不影响策略输出。

8 类编码器各输出 64 维特征，拼接后得到 512 维嵌入，再接入 $[1024, 1024, 1024]$ 的 MLP 主干。actor 输出 12 个动作 logits，critic 输出状态价值。项目同时提供共享 actor-critic 和独立 actor/critic 两种实现，其中 `IndependentNetwork` 更适合在后期训练中获得更稳定的低熵策略。

5.3. 训练算法

训练入口为 `training/ddppo.py`。系统使用 PPO [3] 和 GAE [4] 估计优势，支持 DDP/NCCL 多 GPU 同步训练 [5]。默认超参数包括： $\gamma = 0.999$ 、GAE $\lambda = 0.95$ 、rollout length 为 128、PPO epoch 为 3、clip ratio 为 0.2、学习率为 5×10^{-4} 、entropy coefficient 为 0.01、value loss coefficient 为 0.5。

为了提高训练效率，项目实现了优势过滤。每次 PPO 更新前，系统计算样本优势值并保留 $|\hat{A}|$ 高于阈值的样本；当前实现以本轮最大 $|A|$ 的 1%

作为阈值，并剔除 done 之后的无效样本。该机制使训练更多关注碰撞边缘、路线选择、避障和交互冲突等高信息状态。

6. 实车验证计划

仿真训练只能证明策略在构建的虚拟世界中有效。为了评估 自竞流 在真实车辆上的可迁移性，后续将开展封闭场地低速实车验证。该部分为验证计划，不代表本文已经完成实车测试。

6.1. 测试目标

实车测试目标是验证仿真训练策略在低速真实车辆上的轨迹跟踪、避障、停车、简单交互和安全降级能力。测试将重点观察策略输出在真实底盘延迟、定位误差、控制限幅和传感噪声下是否仍保持稳定。

6.2. 测试阶段

实车验证计划分为三个阶段：

- 阶段一：封闭场地单车低速验证。车辆在限定路线中完成起步、巡航、减速、停车和简单转弯，全程设置速度上限、遥控急停和安全员接管。
- 阶段二：加入静态障碍、锥桶、假车或软目标，验证避障、停车和绕行策略。该阶段不引入真实行人或高风险障碍。
- 阶段三：加入低速动态目标或遥控目标车，验证跟车、让行和简单交互决策。所有动态目标均保持低速，并由外部人员监控。

6.3. 安全约束

测试必须在封闭道路或专用测试场开展。策略输出不直接控制底盘，而是先经过安全控制层、动作限幅器和紧急制动逻辑。测试车辆应配置车内安全员、外部观察员和遥控急停装置；任何定位异常、通信异常、控制异常或人员进入风险区域的情况，都应立即切换人工接管或制动。

6.4. 评估指标

计划记录的指标包括：人工接管次数、碰撞或近碰次数、离路次数、目标到达率、平均速度、加速度和 jerk 舒适性、轨迹跟踪误差、停车误差、策略推理延迟和控制链路总延迟。仿真指标与实车指标将保持同名定义，便于分析 sim-to-real 差距。

7. 结论与展望

本文提出 自竞流 (Selfrace) 框架, 以自博弈强化学习为核心, 将多智能体交互、GPU 批量仿真、结构化局部观测、离散 jerk 控制、条件化奖励和排列不变策略网络整合到一个可扩展的自动驾驶训练系统中。与以原论文为中心的复述不同, 本文从本项目实现出发, 描述了仿真器、地图处理、空间哈希、动力学、奖励、网络和训练闭环的具体设计。

后续工作将沿三条线推进。第一, 继续扩大训练规模, 完善多地图随机化、交通灯和停止线逻辑, 并将车辆动力学随机化参数真正接入策略条件。第二, 建立稳定的闭环评测, 包括自博弈长程事故率、目标到达率、奖励条件消融和路径长度课程实验。第三, 按照本文提出的计划开展封闭场地低速实车验证, 在安全约束下评估策略从仿真到真实车辆的迁移能力。

代码与资料来源

本文以本地 Selfrace 仓库为主要实现依据, 相关模块包括 `configs/default_config.yaml`、`simulator/`、`training/` 和 `utils/spatial_hash.py`。自博弈自动驾驶的大规模训练思想参考文献 [1]。

References

- [1] M. Cusumano-Towner, D. Hafner, A. Hertzberg, B. Huval, A. Petrenko, E. Vinitsky, E. Wijmans, T. Killian, S. Bowers, O. Sener, P. Krähenbühl, V. Koltun, Robust autonomy emerges from self-play (2025). `arXiv:2502.03349`.
URL <https://arxiv.org/abs/2502.03349>
- [2] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, A. Smola, Deep sets, in: Advances in Neural Information Processing Systems, 2017.
URL <https://arxiv.org/abs/1703.06114>
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, in: arXiv preprint `arXiv:1707.06347`, 2017.
URL <https://arxiv.org/abs/1707.06347>
- [4] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation,

in: International Conference on Learning Representations, 2016.
URL <https://arxiv.org/abs/1506.02438>

- [5] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, D. Batra, DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames, in: International Conference on Learning Representations, 2020.
URL <https://arxiv.org/abs/1911.00357>